



DOCUMENTAÇÃO TÉCNICA

Projeto Integrador VI (PI-VI)

ADA Company — Sistema de Gestão para Consultoria em Acessibilidade Digital

1. Identificação do Projeto

1.1. Nome do Projeto: ADA Company — Sistema de Gestão Empresarial para Consultoria em Acessibilidade Digital.

1.2. Grupo/Equipe:

Integrante	Função	RA
Luiz Riato	Product Owner	3011392323011
Matheus Prusch	Desenvolvedor Full-Stack	3011392323025
Maycon Sanches	Desenvolvedor Full-Stack	3011392323013
Pietro Adrian	Desenvolvedor Full-Stack	3011392323005
Samuel Pregnotatto	Desenvolvedor Full-Stack	3011392323042

1.3. Data da Elaboração: 05/06/2026.

1.4. Versão do Documento: 1.0 — 05/06/2026 (referente à versão v0.3.1 do sistema).

1.5. Repositórios do Projeto (GitHub):

- Back-end (NestJS): <https://github.com/ADACompany01/backEnd-SextoSemestre>
- Front-end (React): <https://github.com/ADACompany01/frontEnd-SextoSemestre>

2. Visão Geral do Produto

2.1. Problema a ser Resolvido

Muitas empresas não sabem se seus sites e sistemas digitais estão em conformidade com os padrões internacionais de acessibilidade (WCAG — Web Content Accessibility Guidelines), exigência cada vez mais cobrada por lei e pelo mercado brasileiro. Os principais problemas abordados são:

- **Falta de diagnóstico:** as empresas não têm uma forma simples de medir o nível de acessibilidade dos seus sites.
- **Processo manual e fragmentado:** auditoria, orçamento e contratação de correções normalmente ocorrem em ferramentas e canais separados.
- **Ausência de formalização digital:** a contratação depende de processos físicos ou manuais, sem assinatura digital integrada.

- **Dificuldade de acompanhamento:** o cliente não tem visibilidade do andamento de orçamentos e contratos.

2.2. Solução Proposta

A ADA Company é uma plataforma web full-stack que centraliza todo o ciclo de atendimento em acessibilidade digital. Cada problema acima é endereçado por pelo menos uma expectativa de alto nível:

- **Auditoria automatizada:** análise de acessibilidade de URLs em tempo real via Google Lighthouse, com classificação nos níveis WCAG A, AA e AAA (resolve a falta de diagnóstico).
- **Fluxo unificado:** solicitação → análise → orçamento → contrato em uma única plataforma (resolve a fragmentação).
- **Rastreamento de status:** acompanhamento em tempo real de solicitações, orçamentos e contratos (resolve a dificuldade de acompanhamento).
- **Atendimento assistido:** chatbot “Ada” com árvore de decisão e integração de IA para suporte ao cliente.

2.3. Público-Alvo (Personas)

O sistema atende dois perfis principais de usuário:

- **Cliente (empresa contratante):** empresas que precisam adequar seus sites às normas de acessibilidade. Buscam diagnóstico confiável e contratação simples e digital. Épico: “como gestor de uma empresa, quero descobrir se meu site é acessível e contratar a correção sem burocracia”.
- **Funcionário/Administrador (equipe ADA):** colaboradores que executam auditorias, geram orçamentos e gerenciam contratos. Épico: “como analista da ADA, quero auditar o site do cliente e gerar um orçamento automaticamente a partir do resultado”.

3. Requisitos Funcionais e Não Funcionais

3.1. Requisitos Funcionais (RFs)

ID	Requisito Funcional	Persona
RF01	Cadastro de usuários (clientes e funcionários)	Cliente / Func.
RF02	Autenticação e autorização via JWT com RBAC	Cliente / Func.
RF03	Gestão de pacotes de acessibilidade (A, AA, AAA)	Funcionário
RF04	CRUD completo de clientes (empresas)	Funcionário
RF05	CRUD de funcionários (restrito a administradores)	Funcionário
RF06	Sistema de solicitações e orçamentos de serviço	Cliente / Func.
RF07	Dashboard com visão geral para funcionários	Funcionário
RF08	Comunicação por API REST entre frontend e backend	Cliente / Func.
RF09	Rastreamento de status de orçamentos e contratos	Cliente
RF10	Análise automatizada de acessibilidade via Lighthouse	Cliente / Func.
RF11	Aprovação/rejeição de orçamentos pelo cliente	Cliente
RF12	Histórico de solicitações e fluxo de status	Cliente / Func.
RF13	Atendimento via chatbot Ada (árvore de decisão, IA e voz)	Cliente

3.2. Requisitos Não Funcionais (RNFs)

Cada RNF qualifica um ou mais requisitos funcionais:

ID	Requisito Não Funcional	RF relacionado
RNF01	Tempo de resposta da API inferior a 10 segundos	RF08, RF10
RNF02	Carregamento do dashboard em menos de 10 segundos	RF07
RNF03	Modularidade: módulos independentes e coesos	Todos
RNF04	Hashing de senhas com bcrypt (10 rounds)	RF01, RF02
RNF05	HTTPS com cabeçalhos de segurança (Helmet + CORS)	RF08
RNF06	Autenticação stateless via JWT (expiração de 1 hora)	RF02
RNF07	Controle de acesso baseado em funções (RBAC)	RF02, RF04, RF05
RNF08	Qualidade de código: ESLint, Prettier e cobertura	Todos
RNF09	Clean Architecture com separação de camadas	Todos
RNF10	Responsividade do frontend para diferentes telas	RF07, RF09
RNF11	Conformidade do próprio sistema com WCAG	RF10
RNF12	Logging e tratamento de erros abrangentes	RF12
RNF13	Rate limiting de 100 requisições/minuto por IP	RF02, RF17
RNF14	Portabilidade via Docker (Dockerfile + compose)	Todos
RNF15	Infraestrutura em nuvem (Azure VM + AWS DynamoDB)	Todos

4. Arquitetura do Sistema

4.1. Visão Geral da Arquitetura

O sistema adota arquitetura cliente-servidor, dividida em um frontend SPA (Single Page Application) e um backend RESTful desacoplados, comunicando-se exclusivamente via HTTP/HTTPS em JSON. O backend segue os princípios da Clean Architecture, com separação estrita entre as camadas de domínio, aplicação, infraestrutura e interface.

A Clean Architecture garante que as regras de negócio não dependam de frameworks ou bancos específicos, facilitando testes unitários (casos de uso são classes puras) e permitindo trocar implementações de infraestrutura sem alterar a lógica de negócio. As camadas são:

- **Domain Layer:** entidades de negócio, interfaces de repositórios e regras de domínio, sem dependências externas (src/domain/).
- **Application Layer:** casos de uso, serviços de aplicação e orquestração das regras de negócio (src/application/).
- **Infrastructure Layer:** implementações concretas de banco de dados (Sequelize, Mongoose, DynamoDB), provedores externos e configurações (src/infrastructure/).
- **Interface Layer:** controllers HTTP, DTOs, Guards, Decorators e Swagger (src/interfaces/http/).
- **Modules Layer:** módulos NestJS que agrupam e conectam as camadas (src/modules/).

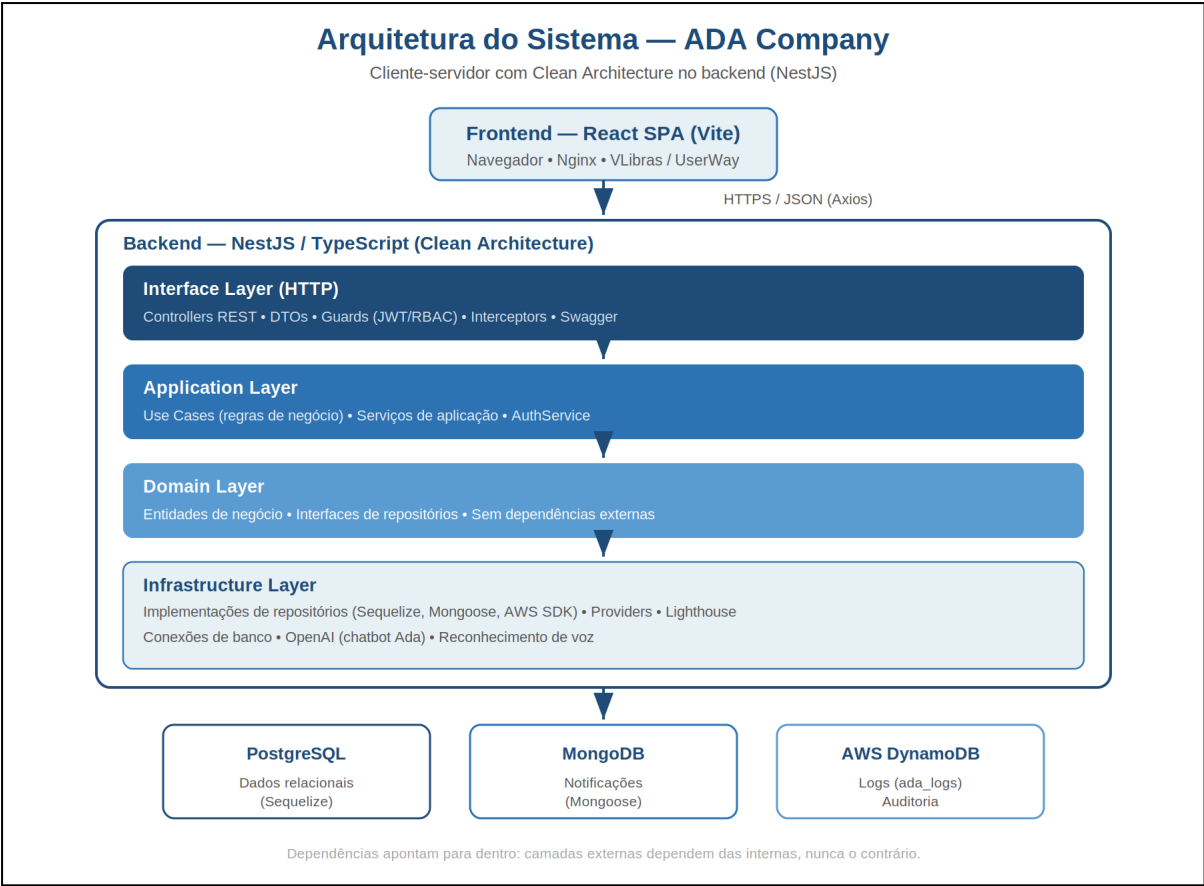


Figura 1 — Arquitetura em camadas (Clean Architecture) do backend e integração com o frontend e os bancos de dados.

4.2. Componentes Principais

O backend é composto por 10 módulos de domínio, cada um seguindo o padrão Repository + Use Cases:

Módulo	Responsabilidade
AuthModule	Login, geração de token JWT e estratégia Passport
ClienteModule	CRUD completo de clientes (empresas)
FuncionarioModule	Gestão de funcionários (acesso restrito a admin)
OrcamentoModule	Criação, edição e exportação de orçamentos em PDF
PacoteModule	Pacotes de serviço (A, AA, AAA) por cliente
SolicitacaoModule	Solicitações de análise de acessibilidade
LighthouseModule	Auditoria de acessibilidade via Google Lighthouse
LogModule	Logging centralizado com DynamoDB e analytics
NotificacaoModule	Sistema de notificações in-app para usuários
ChatbotModule	Ada — chatbot com árvore de decisão + OpenAI GPT

4.3. Tecnologias Utilizadas

Backend

Tecnologia	Versão	Uso
Node.js	20 LTS	Runtime de execução
NestJS	11.1.2	Framework principal do backend
TypeScript	5.1.3	Linguagem tipada
Sequelize	6.37.7	ORM para PostgreSQL
Mongoose	8.x	ODM para MongoDB
AWS SDK	3.x	Integração com DynamoDB
Passport.js	0.7.0	Autenticação com estratégia JWT
bcrypt	5.1.1	Hash seguro de senhas
Lighthouse	12.6.1	Auditoria de acessibilidade web
Puppeteer / Chromium	headless	Renderização para o Lighthouse
class-validator	0.14.2	Validação de DTOs
Swagger / OpenAPI	5.0	Documentação automática da API
@nestjs/throttler	6.4.0	Rate limiting
Helmet	8.1.0	Cabeçalhos HTTP de segurança

Frontend

Tecnologia	Versão	Uso
React	18.2.0	Biblioteca de interface (SPA)
Vite	4.5.14	Build tool e dev server
React Router DOM	6.30.1	Roteamento de páginas
Axios	1.10.0	Cliente HTTP
Bootstrap / React-Bootstrap	5.3.3 / 2.10.5	Estilização
Material-UI	6.1.5	Componentes de interface
jwt-decode	4.0.0	Decodificação de token JWT
VLibras / UserWay	—	Widgets de acessibilidade
Nginx	1.25 (Alpine)	Servidor de produção (container)

Bancos de Dados e DevOps

Tecnologia	Versão	Finalidade
PostgreSQL	15+	Banco relacional principal (dados de negócio)
MongoDB	7.0	Documentos / notificações (via Mongoose)
AWS DynamoDB	—	Logs de sistema (tabela ada_logs)
Docker / Docker Compose	3.8+	Containerização de serviços
GitHub Actions	—	Pipeline CI/CD

Tecnologia	Versão	Finalidade
Jest	29.5.0	Testes unitários (threshold 80%)
Robot Framework	—	Testes E2E de endpoints da API
ESLint + Prettier	8.x / 3.0	Qualidade e padronização de código
OpenAI API	gpt-5.4-nano	Respostas generativas do chatbot Ada

5. Modelo de Dados

5.1. Diagrama de Entidade-Relacionamento (DER)

O banco principal utiliza PostgreSQL com o ORM Sequelize, em modelo relacional normalizado, inicializado via init.sql no container Docker. O DER completo deve ser inserido abaixo:

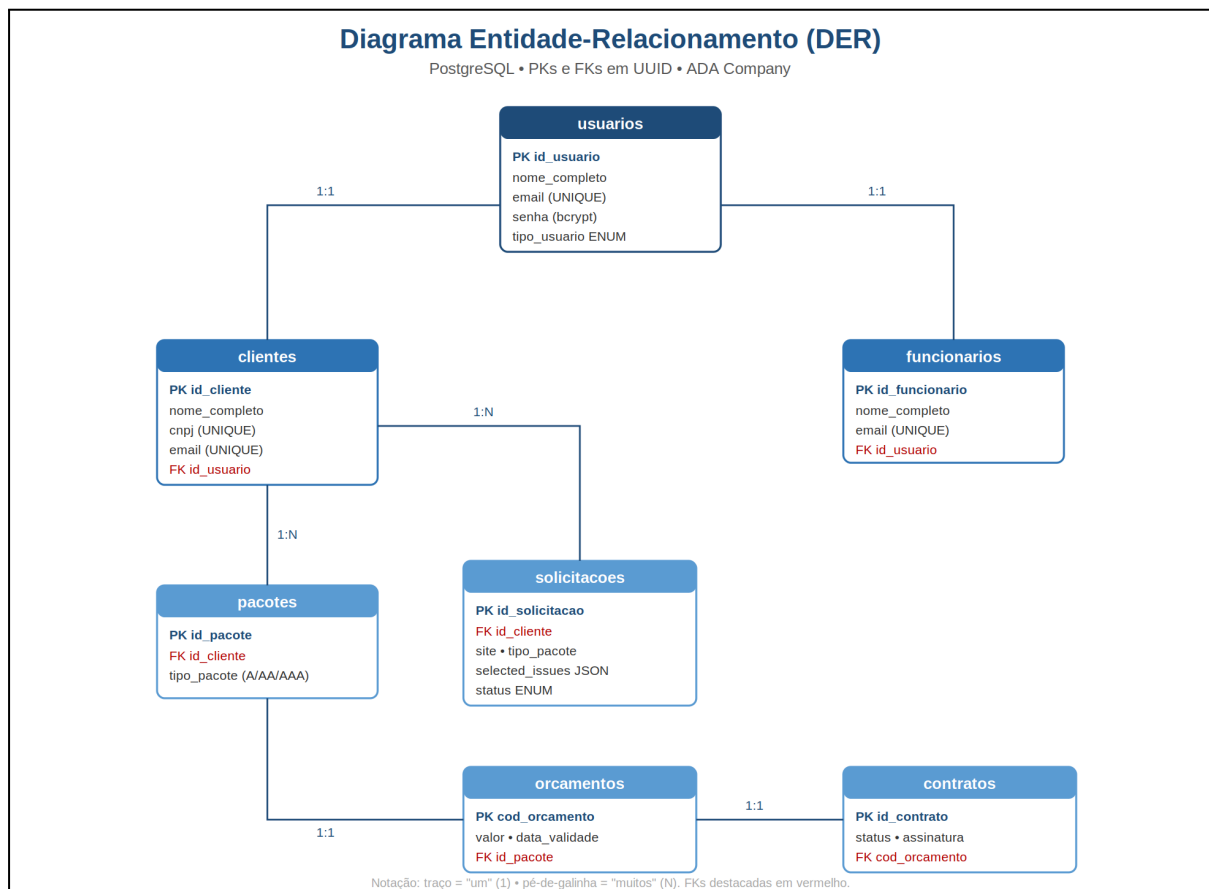


Figura 2 — Diagrama Entidade-Relacionamento do banco PostgreSQL (notação pé-de-galinha).

Resumo dos relacionamentos entre as tabelas:

Origem	Destino	Cardinalidade	Descrição
usuarios	clientes	1:1	Um usuário pode ser um cliente
usuarios	funcionarios	1:1	Um usuário pode ser um funcionário
clientes	pacotes	1:N	Um cliente pode ter vários pacotes
pacotes	orcamentos	1:1	Um pacote gera um orçamento
orcamentos	contratos	1:1	Um orçamento gera um contrato
clientes	solicitacoes	1:N	Um cliente faz várias solicitações

5.2. Descrição das Entidades Principais

usuarios — base de autenticação

Campo	Tipo	Restrição
id_usuario	UUID	PK
nome_completo	VARCHAR	NOT NULL
email	VARCHAR	UNIQUE, NOT NULL
telefone	VARCHAR	NULL
senha	VARCHAR	NOT NULL (bcrypt)
tipo_usuario	ENUM	NOT NULL (funcionario cliente)

clientes / funcionarios

Campo	Tipo	Restrição
id_cliente / id_funcionario	UUID	PK
nome_completo	VARCHAR	NOT NULL
cnpj (apenas clientes)	VARCHAR	UNIQUE, NOT NULL
email	VARCHAR	UNIQUE, NOT NULL
telefone	VARCHAR	NULL
id_usuario	UUID	FK → usuarios

pacotes / orcamentos / contratos / solicitacoes

Entidade	Campos principais	Observações
pacotes	id_pacote (PK), id_cliente (FK), tipo_pacote ENUM(A/AA/AAA), valor_base DECIMAL	Nível WCAG por cliente
orcamentos	cod_orcamento (PK), valor_orcamento, data_orcamento, data_validade, id_pacote (FK), arquivo_orcamento	Validade de 30 dias
contratos	id_contrato (PK), valor_contrato, status_contrato ENUM, data_inicio, data_entrega, cod_orcamento (FK), arquivo_contrato, arquivoassinado	Status
solicitacoes	id_solicitacao (PK), id_cliente (FK), site, tipo_pacote ENUM, observacoes, selected_issues JSON, status ENUM	Fluxo PENDENTE → ... → APROVADO

Nota técnica: PostgreSQL armazena os dados relacionais críticos. MongoDB (via Mongoose) guarda notificações com schema flexível. AWS DynamoDB armazena exclusivamente logs do sistema (tabela `ada_logs`), com consultas por data, usuário e severidade.

6. APIs (Application Programming Interfaces)

6.1. Design da API

A API segue o estilo arquitetural REST (RESTful), com comunicação sobre HTTP/HTTPS e payloads em JSON. Todos os endpoints são documentados via Swagger/OpenAPI, na rota `/api`

do backend. A URL base de produção é <http://apiadacompany.duckdns.org/api>. Endpoints representativos por recurso:

Método	Rota	Função	Acesso
POST	/auth/login	Login do usuário (retorna JWT)	Pública
POST	/clientes/cadastro	Cadastro de novo cliente	Pública
GET	/clientes	Listar todos os clientes	Funcionário
GET	/clientes/me	Dados do cliente logado	Cliente
POST	/pacotes	Criar pacote de serviço	Funcionário
POST	/solicitacoes	Criar solicitação de análise	Cliente
POST	/solicitacoes/{id}/orcamento	Gerar orçamento da solicitação	Funcionário
POST	/orcamentos	Criar orçamento	Funcionário
POST	/contratos	Criar contrato	Funcionário
POST	/contratos/{id}/sign	Assinar contrato digitalmente	Cliente
POST	/mobile/lighthouse/analyze	Analisar acessibilidade de site	Autenticado
GET	/chatbot/tree	Obter árvore de decisão do chatbot	Pública
POST	/chatbot/llm	Resposta generativa da Ada (OpenAI)	Pública
GET	/logs	Listar logs com filtros	Funcionário
GET	/notificacoes/{userId}	Notificações do usuário	Próprio

Nota técnica: A tabela resume endpoints representativos. A API expõe dezenas de rotas REST no total, todas no Swagger, cobrindo as operações CRUD de cada recurso (clientes, funcionários, pacotes, solicitações, orçamentos, contratos, logs e notificações).

6.2. Autenticação e Autorização

A autenticação usa JWT (JSON Web Token), em modelo stateless. O fluxo é:

- **Login:** o usuário envia e-mail e senha para POST /auth/login; o sistema valida no PostgreSQL e compara a senha com bcrypt.
- **Token:** em caso de sucesso, retorna um JWT assinado (expiração de 1 hora) com id, e-mail e tipo de usuário.
- **Requisições:** o frontend envia o token no header Authorization: Bearer <token> em cada requisição protegida.
- **Autorização (RBAC):** Guards verificam token e papel — JwtAuthGuard (rotas protegidas), FuncionarioGuard (admin) e SelfAccessGuard (dono do recurso).

7. Computação em Nuvem (Azure)

7.1. Serviços Azure Utilizados

O ambiente de produção ativo do projeto é uma Azure Virtual Machine (Azure VM) que hospeda toda a aplicação via Docker. Os recursos de nuvem empregados são:

- **Azure Virtual Machine:** VM Linux que executa o Docker Engine e hospeda os três containers da aplicação (frontend, backend e PostgreSQL).

- **Docker Hub:** registro de imagens onde as imagens de backend e frontend são publicadas pelo pipeline.
- **MongoDB Atlas (provável):** banco de documentos acessado via MONGODB_URI (secret), indicando serviço gerenciado externo.

7.2. Arquitetura de Deploy

O deploy é automatizado por um pipeline CI/CD no GitHub Actions, acionado a cada push na branch main. A VM executa três containers Docker conectados pela rede interna ada-net:

Container	Imagem	Porta (VM)	Restart
postgres-db	postgres:latest	Interno (sem expose)	always
api-backend	{DOCKER_HUB}/ada-company-backend-mobile:latest	3001	always
web-frontend	{DOCKER_HUB}/ada-company-frontend:latest	8080 (→ 80)	always

O backend conecta-se ao PostgreSQL pelo nome do container na rede ada-net (DNS interno do Docker), via DB_HOST=postgres-db — não localhost. O PostgreSQL não expõe porta para fora da VM, sendo acessível apenas internamente pelos containers da mesma rede. O Chromium usado pelo Lighthouse é configurado por variáveis de ambiente (CHROME_PATH e PUPPETEER_EXECUTABLE_PATH = /usr/bin/chromium).

Etapas do pipeline (push na main):

1. **Job 1 — Build e Teste:** npm install, ESLint, testes unitários (Jest), verificação de cobertura (80%), build NestJS e upload do artefato dist/.
2. **Job 2 — Versionamento e Deploy:** cálculo da versão semântica (BREAKING CHANGE → major, feat → minor, demais → patch), commit do VERSION.txt e tag Git, build e push da imagem Docker com três tags (:latest, :<branch>, :<versão>).
3. **Deploy via SSH:** o pipeline acessa a Azure VM por SSH e executa docker image prune, cria a rede ada-net (se não existir), remove os containers antigos e sobe postgres-db + api-backend. Em falha, envia e-mail de notificação.

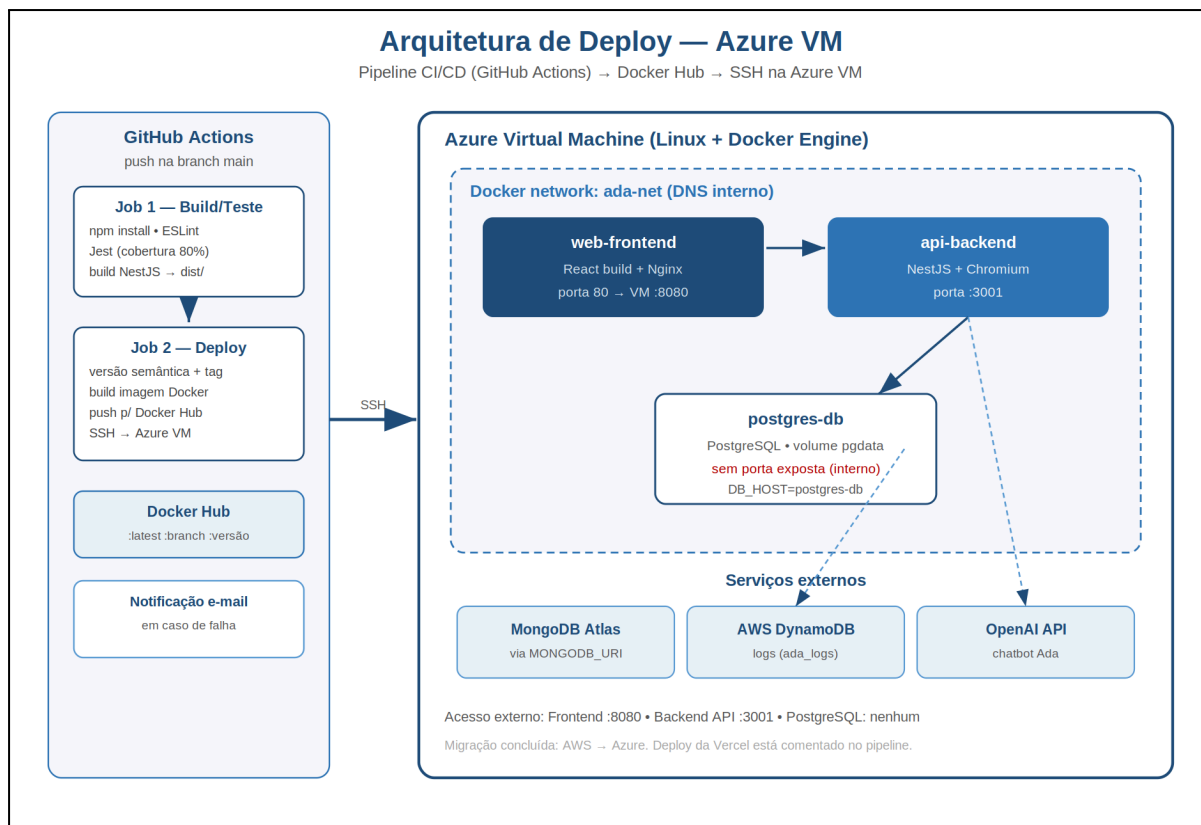


Figura 3 — Arquitetura de deploy: pipeline CI/CD, Docker Hub e containers na Azure VM (rede ada-net).

7.3. Monitoramento e Logs

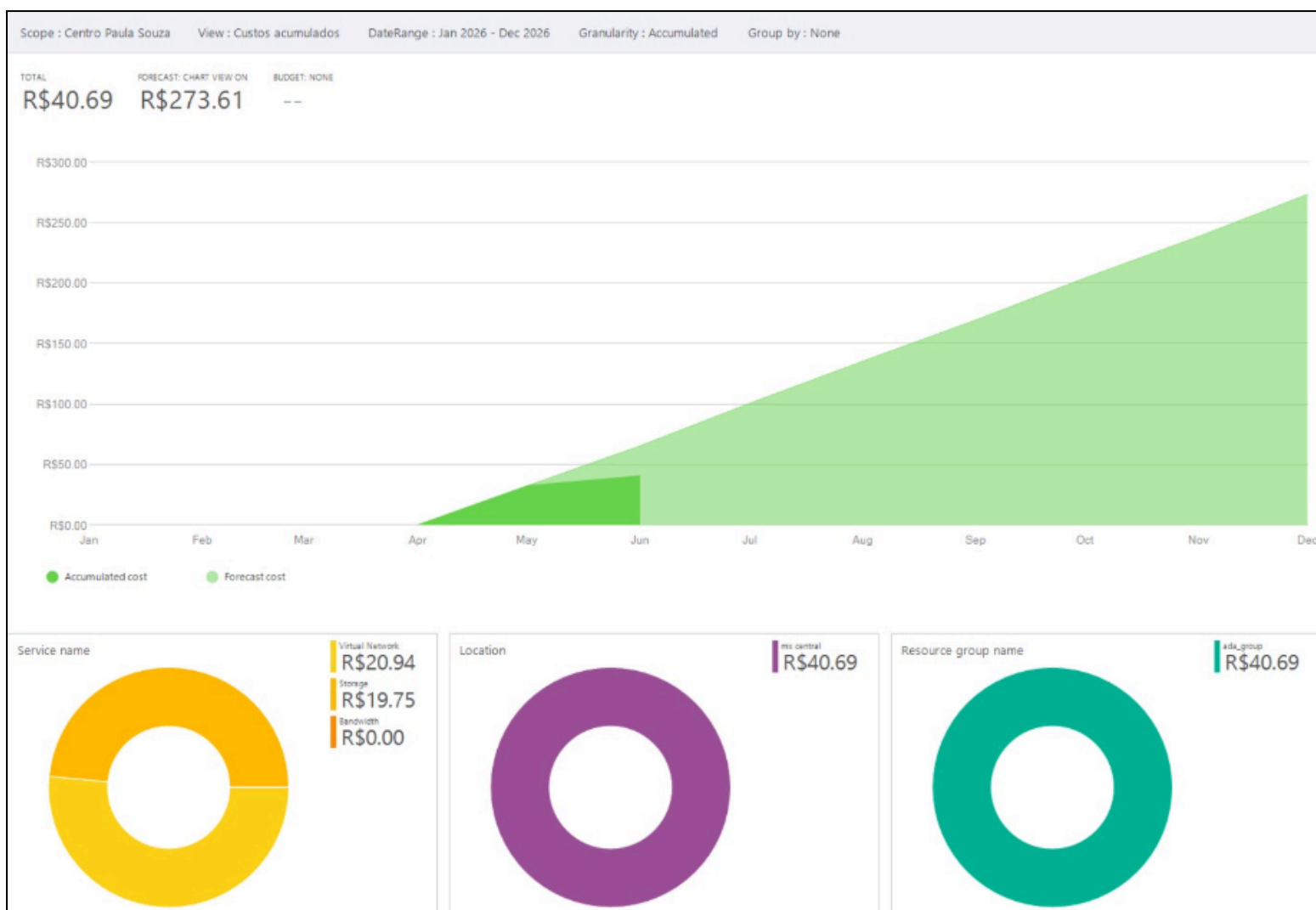
O monitoramento e a auditoria do sistema são feitos por meio de:

- **Logging centralizado:** um interceptor registra as operações da API no Azure, com filtros por data, usuário e severidade, estatísticas de uso e limpeza automática de logs antigos.
- **Health checks:** os containers possuem health check; o módulo Lighthouse expõe o endpoint /mobile/lighthouse/health.
- **Notificação por e-mail:** o pipeline envia e-mail automático em caso de falha no build ou no deploy.

7.4. Estimativa de Custos

Os custos de infraestrutura em nuvem do projeto foram apurados diretamente no Azure Cost Management, no escopo da assinatura institucional (Centro Paula Souza), resource group ada_group, região ms central.

No período acumulado de janeiro a junho de 2026, o custo real total da infraestrutura Azure foi de R\$ 40,69. Com base nesse consumo, o Azure projeta um custo acumulado de R\$ 273,61 até dezembro de 2026 (previsão para o ano completo). A distribuição por serviço esta na imagem abaixo:



8. Qualidade e Testes de Software

8.1. Estratégia de Testes

A qualidade é garantida por uma combinação de testes automatizados executados localmente e no pipeline de CI/CD:

- **Testes unitários:** cobrem os casos de uso da camada de aplicação (classes puras, isoladas via injeção de dependência e mocks de repositório).
- **Testes E2E / integração:** validam os endpoints HTTP completos da API.
- **Análise estática:** ESLint e Prettier garantem padronização e detectam problemas antes da execução, como etapa obrigatória do pipeline.

8.2. Ferramentas de Teste

Tipo	Ferramenta	Localização / Escopo
Unitários	Jest 29.5.0 + ts-jest	src/**/*.spec.ts (use-cases)
E2E (backend)	Jest E2E + Supertest 7.0.0	test/jest-e2e.json
API / Integração	Robot Framework	robot/tests/ (endpoints do chatbot)

Tipo	Ferramenta	Localização / Escopo
Frontend	Vitest 0.34.6 + Testing Library	Ambiente JSDOM
Qualidade	ESLint + Prettier	Backend e frontend

8.3. Cobertura de Testes

A meta de cobertura é de no mínimo 80% de cobertura de código nos casos de uso do backend, verificada automaticamente pelo pipeline (threshold do Jest). A cobertura do frontend é monitorada via relatório do Vitest.

9. Processamento de Linguagem Natural (PLN)

9.1. Funcionalidades de PLN

As funcionalidades de PLN do sistema concentram-se no chatbot Ada, assistente virtual de suporte ao cliente disponível antes e após o login, complementado por reconhecimento de voz. Suas capacidades:

- **Reconhecimento de voz (speech-to-text):** o usuário pode interagir por voz, com a fala transcrita em texto para ser processada pelo sistema.
- **Árvore de decisão:** fluxo predefinido que guia o usuário por opções estruturadas (GET /chatbot/tree).
- **Respostas generativas:** integração com a OpenAI API para responder em linguagem natural (POST /chatbot/llm), com contexto dos serviços da ADA injetado nas respostas.
- **Fallback local:** quando a API externa está indisponível, recorre a um processamento de NLP local.
- **Controle de uso:** limite diário padrão de 50 chamadas à OpenAI e histórico de conversa por sessão.

9.2. Tecnologias e Modelos

Recurso	Detalhe
Provedor de IA	OpenAI API
Modelo padrão	gpt-5.4-nano (configurável via variável de ambiente)
Limite de tokens de saída	300 tokens (configurável)
Limite diário	50 chamadas (configurável)
Abordagem complementar	Árvore de decisão + fallback de NLP local
Reconhecimento de voz	Speech-to-text

9.3. Dataset e Treinamento

Nem o chatbot Ada nem o reconhecimento de voz treinam um modelo de PLN próprio. O chatbot consome um modelo de linguagem pré-treinado da OpenAI (gpt-5.4-nano) via API, complementado por uma árvore de decisão construída manualmente e por contexto de domínio injetado em tempo de execução (prompt). O reconhecimento de voz utiliza um serviço/biblioteca de speech-to-text já treinado. Portanto, não há dataset de treinamento próprio, etapa de fine-tuning nem métricas de treinamento (acurácia, F1-score) aplicáveis a este projeto.

Nota técnica: Não foram inventadas métricas de treinamento porque o projeto não treina modelo próprio — usa serviços pré-treinados (OpenAI para texto e speech-to-text para voz). A qualidade das respostas depende desses serviços e da engenharia de prompt. Caso o grupo deseje avaliar, recomenda-se uma medição qualitativa da adequação das respostas e da precisão da transcrição em um conjunto de exemplos-teste.

10. Integração entre Componentes

10.1. Fluxo de Dados

O fluxo de dados percorre os componentes refletindo o fluxo de atendimento de negócio:

1. O cliente acessa o frontend (React SPA), que se comunica com a API REST via Axios sobre HTTPS.
2. A API (NestJS) processa a requisição nas suas camadas, persistindo dados relacionais no PostgreSQL.
3. Solicitações de análise acionam o LighthouseModule, que executa o Chrome headless sobre a URL e retorna os scores de acessibilidade.
4. Notificações são gravadas no MongoDB; logs de cada requisição são enviados ao DynamoDB pelo interceptor.
5. O chatbot Ada, quando acionado, consulta a OpenAI API e retorna a resposta ao frontend.

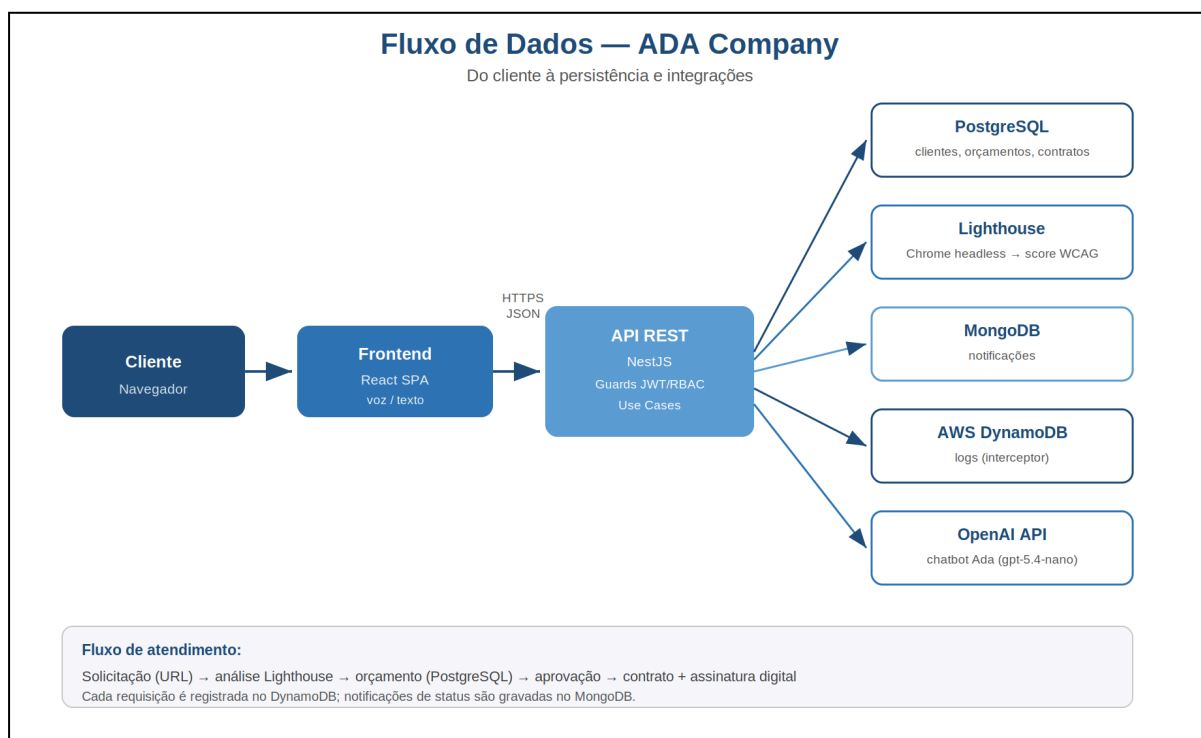


Figura 4 — Fluxo de dados entre o cliente, a API e os serviços de persistência e integração.

10.2. Contratos de Interface

Os componentes se comunicam por contratos bem definidos:

- **Formato de dados:** JSON em todas as requisições e respostas da API REST.
- **Protocolo:** HTTP/HTTPS, com autenticação via header Authorization: Bearer <JWT>.
- **Validação de contrato:** DTOs validados por class-validator garantem que apenas dados no formato esperado entrem na aplicação.

- **Documentação do contrato:** o Swagger/OpenAPI descreve formalmente cada endpoint, seus parâmetros e respostas.

11. Riscos e Mitigações

Riscos técnicos identificados e seus respectivos planos de mitigação (itens 11.1 e 11.2 consolidados):

Risco	Impacto	Mitigação
Bloco de deploy duplicado no ci-backend.yml (linhas 240-284 e 285-321); o bloco ativo faz bind em 127.0.0.1:3001:3001 (apenas localhost)	Alto — a API pode ficar inacessível externamente sem proxy reverso, quebrando a comunicação com o frontend	Remover o bloco duplicado/quebrado e padronizar o bind de porta; validar acesso externo após o deploy ou configurar proxy reverso (ex.: Nginx) na VM
Segredos expostos em versões anteriores da documentação (JWT_SECRET, credenciais)	Alto — comprometimento de autenticação e dados	Manter todos os segredos apenas em GitHub Secrets / .env; rotacionar quaisquer chaves já expostas
Ponto único de falha: toda a aplicação roda em uma única Azure VM	Médio — indisponibilidade total se a VM cair	Configurar backups/snapshots da VM e do volume pgdata; avaliar redundância ou serviço gerenciado
Dependência de cota da OpenAI para o chatbot	Baixo/Médio — perda da resposta generativa	Já há fallback de NLP local e limite diário; monitorar consumo
Tag :latest nas imagens Docker dificulta rollback preciso	Médio — difícil reverter a uma versão estável específica	Usar a tag de versão semântica (:<versão>) gerada pelo pipeline para deploy e rollback
PostgreSQL em container com postgres:latest (versão não fixada)	Médio — atualização inesperada pode quebrar compatibilidade	Fixar a versão da imagem (ex.: postgres:15) no compose/deploy

12. Itens do Product Backlog (PBI)

Os itens do Product Backlog são geridos no board Kanban da equipe no Trello (link no Anexo 13.1). A tabela a seguir consolida os itens conforme o estado atual do board, organizados pelo fluxo de trabalho (do concluído ao pendente):

Status	Item do Backlog (PBI)
Concluído	Sprint 1 — Testes e Reconhecimento de Voz (7/7); Pipeline de integração de testes (GitHub Actions); Reconhecimento de voz; Teste unitário; Documentação técnica do plano de testes; reuniões de alinhamento e de planejamento de sprint
Aceite do PO	Chatbot de atendimento ao cliente (Ada) — em validação pelo Product Owner
Concluído	Sprint 2 — Migração e Documentação (8–22 mai); Migração AWS → Azure (Marco 1); Documentação do Usuário; Documentação de Desenvolvimento; Documentação Técnica
A fazer	Teste de Caixa Preta; Teste de Caixa Cinza; Teste de Caixa Branca

13. Anexos

13.1. Backlog do Produto

O backlog do produto é mantido no board Kanban da equipe no Trello:

<https://trello.com/b/ORIn6qoA/kanban-ada-company-6sem> (KANBAN ADA COMPANY 6ºSEM).

13.2. Repositórios de Código-Fonte O código-fonte do projeto está versionado no GitHub, distribuído em dois repositórios:

- Back-end (NestJS): <https://github.com/ADACompany01/backEnd-SextoSemestre> — contém também a pasta `doctos` com as três documentações em PDF.
- Front-end (React): <https://github.com/ADACompany01/frontEnd-SextoSemestre>

13.3. Referências

Documentação técnica e referências do modelo (formato ABNT):

- AGUIAR, F.; CAROLI, P. Product Backlog Building: um guia prático para criação e refinamento de backlog para produtos de sucesso. Rio de Janeiro: Editora Caroli, 2021.
- DYNOWSKI, L.; DULAK, M. Dominando estilos de APIs: compreendendo as vantagens e desvantagens dos principais estilos de APIs e escolhendo as soluções corretas. São Paulo: Novatec, 2025.
- NESTJS. Documentation. Disponível em: <https://docs.nestjs.com>. Acesso em: 5 jun. 2026.
- GOOGLE. Lighthouse — Accessibility. Disponível em: <https://developer.chrome.com/docs/lighthouse>. Acesso em: 5 jun. 2026.
- W3C. Web Content Accessibility Guidelines (WCAG) 2.1. Disponível em: <https://www.w3.org/TR/WCAG21>. Acesso em: 5 jun. 2026.

13.4. Ferramentas de Inteligência Artificial Utilizadas

Em conformidade com a solicitação de transparência sobre o uso de IA, declara-se que as seguintes ferramentas foram empregadas como apoio durante o desenvolvimento e a documentação do projeto:

Claude (Anthropic) — assistente de IA utilizado via interface de chat para apoio à elaboração e revisão das três documentações (técnica, de desenvolvimento e de usuário), estruturação de conteúdo conforme o modelo da disciplina, revisão de clareza e consistência textual, e organização das informações fornecidas pela equipe.

Claude Code (Anthropic) — extensão do Claude integrada ao Visual Studio Code, utilizada para leitura e análise do código-fonte do projeto nos repositórios (front-end e back-end), apoiando a verificação da arquitetura, das tecnologias empregadas e a coerência entre a implementação e a documentação técnica.

As ferramentas foram utilizadas como apoio, sob direção e revisão da equipe, que forneceu os dados do projeto (papéis, arquitetura, backlog, custos e demais informações reais) e validou todo o conteúdo gerado.

Registro do prompt utilizados (Resumido)

Documentação Técnica:

"Me ajude a organizar a documentação técnica do ADA Company seguindo o modelo, a partir destes dados de arquitetura: back-end NestJS (Clean Architecture), front-end React, bancos PostgreSQL/MongoDB/DynamoDB, auditoria via Google Lighthouse, deploy em Azure VM com Docker e CI/CD no GitHub Actions. Inclua DER, requisitos, descrição do módulo de PLN e a estimativa de custos de nuvem."
